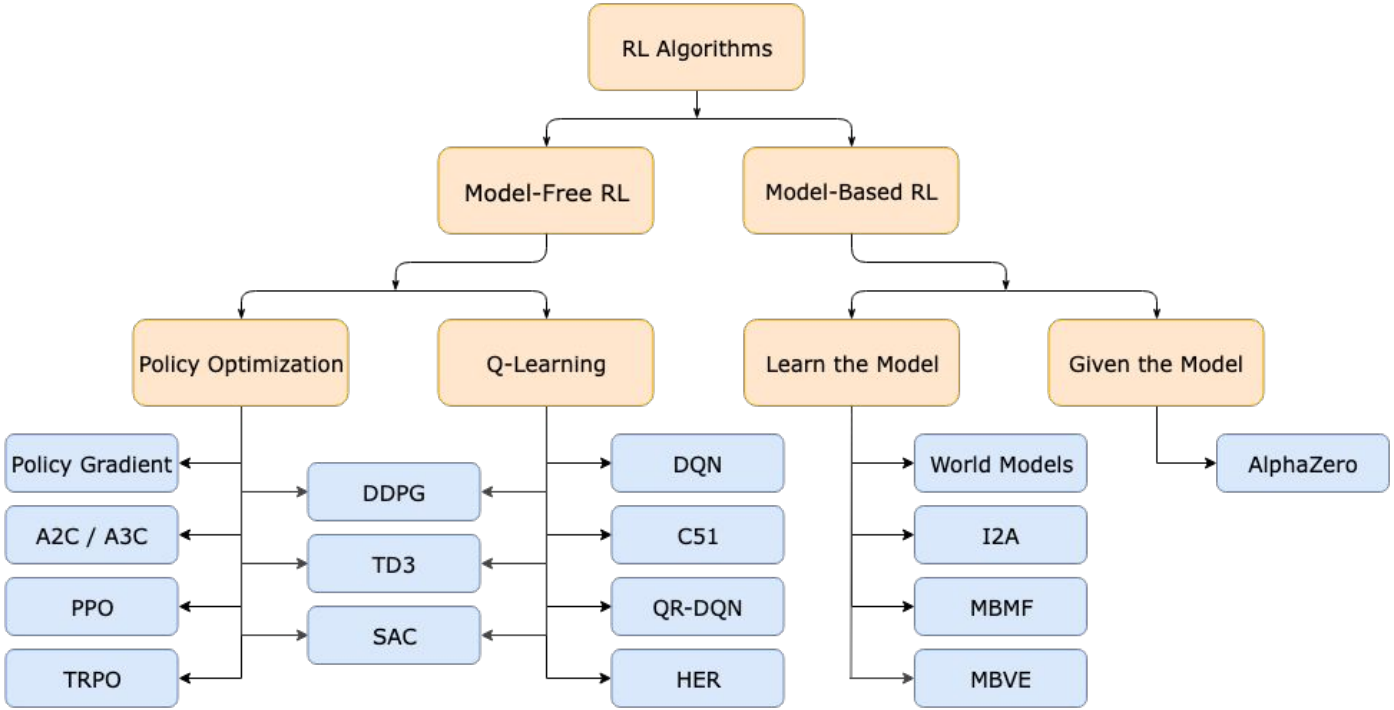
Policy Gradient Methods

Types of RL Algorithms



https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

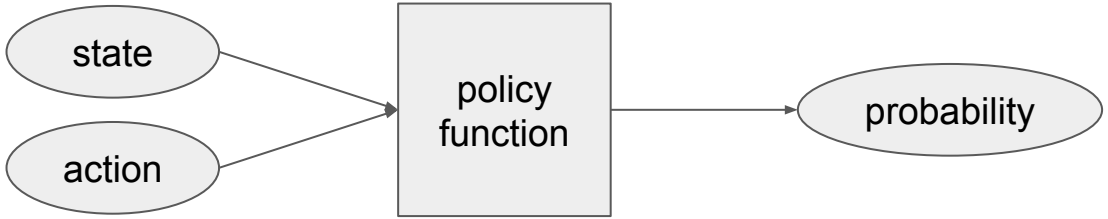
Types of RL Algorithms

- So far, we have covered so-called “Action-value” methods (e.g. Q-learning)
- They learned the values of actions and then selected actions based on their estimated action value
- Is there any other way?
- Yes, policy gradient methods!
- Learn a <u>parameterized</u> policy that can select actions without consulting a value function

Parameterized Policy

- What is a policy?
- A policy is a function to map a state-action pair to a probability to choose the action at the state
- Can we approximate the policy function itself?

$$\pi(a|s, \boldsymbol{\theta}) = \Pr\{A_t = a \mid S_t = s, \boldsymbol{\theta}_t = \boldsymbol{\theta}\}$$



Policy Gradient Methods: General Schema

1. Define <u>a performance function</u> $J(\theta)$ that indicates the performance of <u>the policy</u> parameterized by θ

2. Solve a maximization problem of the performance function by taking Gradient <u>ascent</u> steps

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)}$$

How do you solve a maximization problem?

- A simple mathematical solution

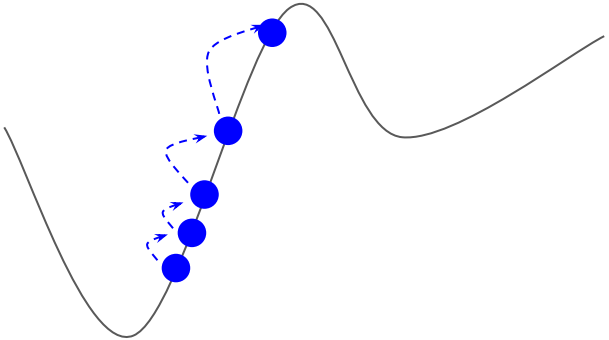
$$J(w) = -w^2$$

- Differentiate J with respect to w
- It is not always possible with a complex objective function
- Also, it tends to be computationally expensive

How do you solve a maximization problem?

- Gradient Ascent (GA)
  - Starting from an initial point $\mathbf{w}_0$
  - Always find another point $\mathbf{w}_{t+1}$ that the corresponding function value increases, compared to the one at the previous point $\mathbf{w}_t$

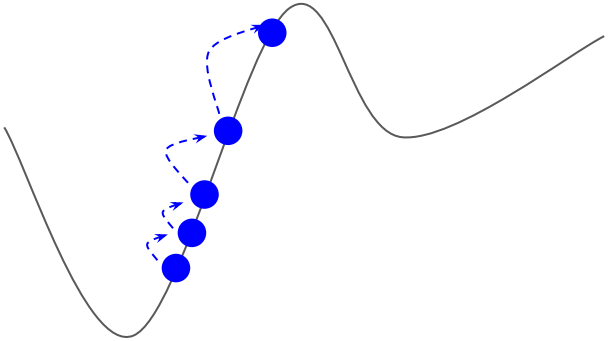
$$J(w_{t+1}) \geq J(w_t)$$



How do you solve a maximization problem?

- The gradient $\nabla J(\mathbf{w})$ of a scalar-valued differentiable function J at a point $\mathbf{w}$

$$\nabla J(\mathbf{w}) = \begin{bmatrix} \frac{\partial J}{\partial w_1}(\mathbf{w}) \\ \vdots \\ \frac{\partial J}{\partial w_n}(\mathbf{w}) \end{bmatrix}$$



Milestone Questions: Gradient

Compute the gradient of the following function

$$J(w_1, w_2) = -w_1^2 + w_1 w_2$$

Agenda

1. How to parameterize a policy function
   - Any parameterization should work as long as the function π is differentiable with respect to its parameters
   - Two examples of the parameterization
     - Discrete action space: Softmax in action preferences
     - Continuous action space: Gaussian distribution
2. How to define a policy performance function J

Softmax in action preferences

- Assume: the action space is discrete and not too large
- Define parameterized numerical preferences (a larger value indicates that the state action pair is more preferred)

$$h(s, a, \boldsymbol{\theta})$$

- We compute the corresponding probabilities based on the preferences, using the softmax function

$$\pi(a|s, \boldsymbol{\theta}) \doteq \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

Milestone Question: An example

- Two actions: a1 and a2
- h(s, a1, theta) = 10
- h(s, a2, theta) = 2
- Compute the probability of choosing a1

$$\pi(a|s,\boldsymbol{\theta}) \doteq \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

Milestone Question: An example

- Two actions: a1 and a2
- h(s, a1, theta) = 10
- h(s, a2, theta) = 2
- Compute the probability of choosing a1
- exp(10) / [exp(10) + exp(2)] = 0.9996646499

$$\pi(a|s,\boldsymbol{\theta}) \doteq \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

Softmax in action preferences

- The numerical preferences can be parameterized arbitrarily
  - Neural network
  - A linear combination of features
  - …
- Here, for simplicity, we look at the linear function case

$$h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{x}(s, a)$$

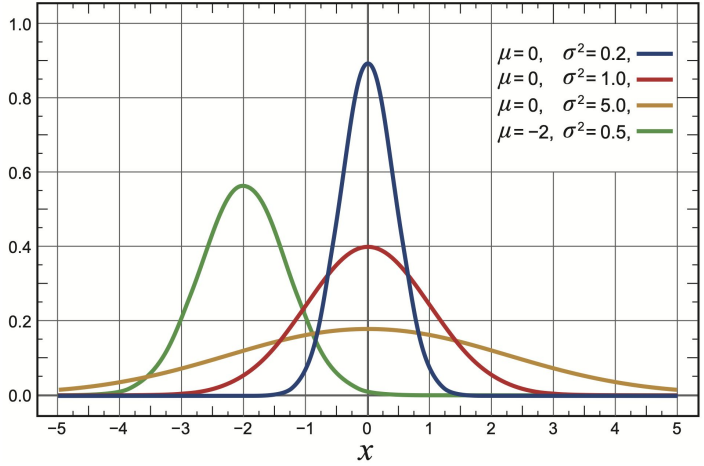
x(s, a) is a feature representation of a state-action pair (s, a)

Gaussian distribution

- Instead of computing learned probabilities for each of the many actions, we can learn statistics of the probability distribution
- For example, when we can assume that there is a set of similar actions that perform well, we can use a Gaussian distribution
- The probability density function

$$p(x) \doteq \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- Here, only factors to examine are mean and standard deviation



Note: the y-axis shows the “density” not probability. The area under a curve sums to 1.0.

Gaussian distribution

- Therefore, the parametric policy function is

$$\pi(a|s,\boldsymbol{\theta}) \doteq \frac{1}{\sigma(s,\boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{(a-\mu(s,\boldsymbol{\theta}))^2}{2\sigma(s,\boldsymbol{\theta})^2}\right)$$

- In a parameter vector, one part to be used for the approximation of the mean and one part for the approximation of the standard deviation

$$\boldsymbol{\theta} = [\boldsymbol{\theta}_\mu, \boldsymbol{\theta}_\sigma]^\top$$

- e.g., The mean can be approximated as a linear function; The standard deviation must always be positive and is better approximated as the exponential of a linear function

$$\mu(s,\boldsymbol{\theta}) \doteq \boldsymbol{\theta}_\mu{}^\top \mathbf{x}_\mu(s) \quad \text{and} \quad \sigma(s,\boldsymbol{\theta}) \doteq \exp\left(\boldsymbol{\theta}_\sigma{}^\top \mathbf{x}_\sigma(s)\right)$$

Milestone Question: An example

- $\theta_\mu = [1, 1]$
- $\theta_\sigma = [0.5, 1]$
- $\mathbf{x}_\mu(s_1) = [1, 2]$
- $\mathbf{x}_\sigma(s_1) = [0, 1]$
- Compute the probability of choosing a = 1.0

$$\pi(a|s, \boldsymbol{\theta}) \doteq \frac{1}{\sigma(s, \boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{(a - \mu(s, \boldsymbol{\theta}))^2}{2\sigma(s, \boldsymbol{\theta})^2}\right)$$

$$\mu(s, \boldsymbol{\theta}) \doteq \boldsymbol{\theta}_\mu{}^\top \mathbf{x}_\mu(s)$$

$$\sigma(s, \boldsymbol{\theta}) \doteq \exp\left(\boldsymbol{\theta}_\sigma{}^\top \mathbf{x}_\sigma(s)\right)$$

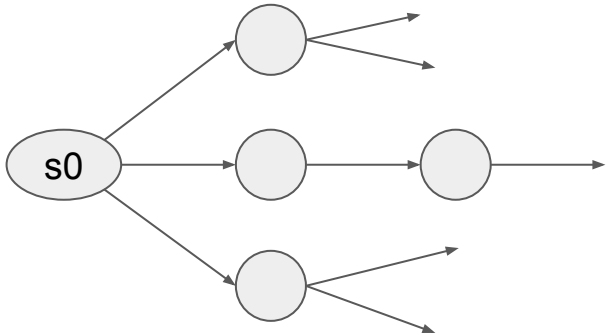
Agenda

1. How to parameterize a policy function
   - Any parameterization should work as long as the function π is differentiable with respect to its parameters
   - Two examples of the parameterization
     - Discrete action space: Softmax in action preferences
     - Continuous action space: Gaussian distribution
2. How to define a policy performance function J

Policy Performance Function J

- In the <u>episodic</u> case, we can define the performance as

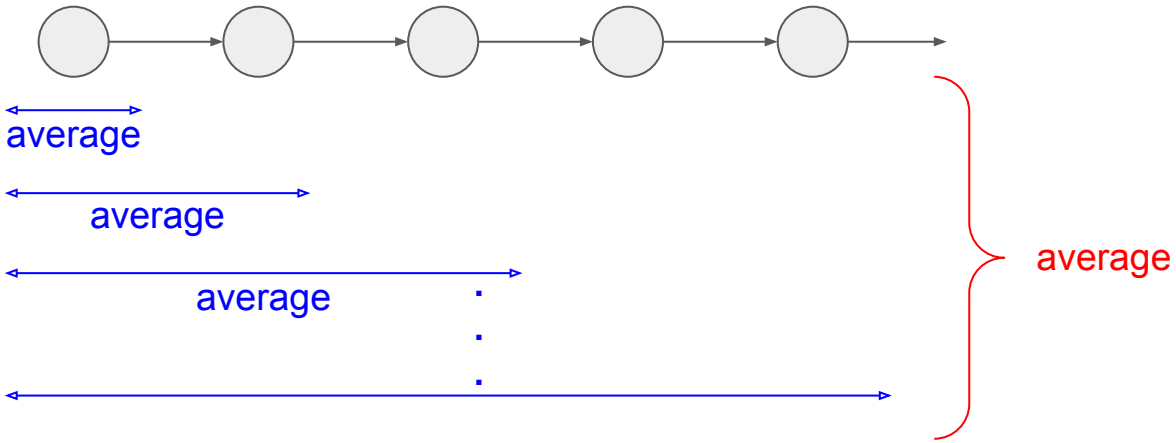
$$J(\boldsymbol{\theta}) \doteq v_{\pi_{\boldsymbol{\theta}}}(s_0)$$



Policy Performance Function J

- In the <u>continuing</u> case, we can think about the concept of average reward of each time step t

$$J(\boldsymbol{\theta}) \doteq r(\pi) \doteq \lim_{h\to\infty} \frac{1}{h} \sum_{t=1}^{h} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi]$$



Summary

- Your policy is parameterized by a variable vector
- If you change the variable vector, the policy changes
- When policy changes, the expected returns (value function) changes

→ Change the variable vector so the expected returns will be maximized

Policy Performance Function J

- In an ergodic MDP,
  - where the steady-state distribution $\mu(s) \doteq \lim_{t\to\infty} \Pr\{S_t = s | A_{0:t} \sim \pi\}$ exists and is independent of S0
  - i.e., the starting state and any early decision made by the agent can have only a temporary effect; in the long run the expectation of being in a state depends only on the policy and the MDP transition probabilities

$$
\begin{aligned}
J(\boldsymbol{\theta}) \doteq r(\pi) &\doteq \lim_{h\to\infty} \frac{1}{h} \sum_{t=1}^{h} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\
&= \lim_{t\to\infty} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\
&= \sum_{s} \mu(s) \sum_{a} \pi(a|s) \sum_{s',r} p(s', r | s, a) r
\end{aligned}
$$

The Policy Gradient Theorem

- Provides an analytic expression for the gradient of performance J with respect to the policy parameter θ
- For the episodic case,

$$\nabla J(\boldsymbol{\theta}) \propto \sum_{s} \mu(s) \sum_{a} q_\pi(s,a) \nabla \pi(a|s,\boldsymbol{\theta})$$

"proportional to"

$$= \mathbb{E}_\pi \left[\sum_{a} q_\pi(S_t,a) \nabla \pi(a|S_t,\boldsymbol{\theta}) \right]$$

- When sampling episodes, the distribution of state visits should be represented in the simple average based on a current policy

REINFORCE: Monte Carlo Policy Gradient

- Similarly, we can replace the sum over all actions a with a sampled At in an experience

$$
\begin{aligned}
\nabla J(\boldsymbol{\theta}) &\propto \mathbb{E}_\pi\left[\sum_a \pi(a|S_t,\boldsymbol{\theta}) q_\pi(S_t,a) \frac{\nabla\pi(a|S_t,\boldsymbol{\theta})}{\pi(a|S_t,\boldsymbol{\theta})}\right] \\
&= \mathbb{E}_\pi\left[q_\pi(S_t,A_t)\frac{\nabla\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})}\right] && \text{(replacing } a \text{ by the sample } A_t \sim \pi) \\
&= \mathbb{E}_\pi\left[G_t\frac{\nabla\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})}\right], && \text{(because } \mathbb{E}_\pi[G_t|S_t,A_t] = q_\pi(S_t,A_t))
\end{aligned}
$$

REINFORCE: Monte Carlo Policy Gradient

- Based on the previous derivation of $\nabla J(\theta)$,

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t|S_t, \boldsymbol{\theta}_t)}{\pi(A_t|S_t, \boldsymbol{\theta}_t)}$$

- The update increases the parameter vector in this direction
  - proportional to the return (it causes the parameter to move most in the directions that favor actions that yield the highest return)
  - inversely proportional to the action probability (otherwise actions that are selected frequently are at an advantage [the updates will be more often in their direction] and might win out even if they do not yield the highest return)

REINFORCE: Monte Carlo Policy Gradient

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Equivalent to $\frac{\nabla \pi(A_t|S_t, \boldsymbol{\theta}_t)}{\pi(A_t|S_t, \boldsymbol{\theta}_t)}$ (It is called an *eligibility vector*: $\nabla \ln x = \frac{\nabla x}{x}$)

It is an MC method since the parameters are updated based on samples of returns from full episodes

Advantages of Policy Approximation

1. The policy may be a simpler function to approximate
   - Action-value methods may overcomplicate the approximation by estimating individual Q values

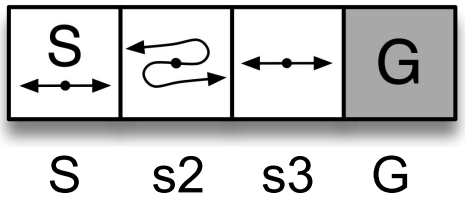
2. The softmax enable the selection of actions with arbitrary probabilities (next example!)
   - An optimal policy could be stochastic
   - Action-value methods cannot produce it
   - E.g. Bluffing in poker; Rock-Paper-Scissors

Advantages of Policy Approximation

3. You can inject prior knowledge about the desired form of the policy
   - Initialization and the function form can provide a bias (in a good way if designed well)

4. Stronger convergence guarantees are available
   - A series of intermediate policies show the continuity

Example: Short corridor with switched actions



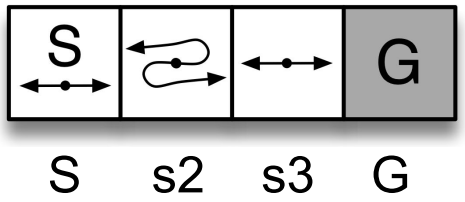
- The reward is -1 per step
- There are only two actions, right and left
  - S & s3: Usual deterministic consequences (left causes no movement in s1)
  - s2: Actions are reversed (right moves to the left and left moves to the right)
- Feature representation of states is not well designed and all the states appear identical; for all states s,

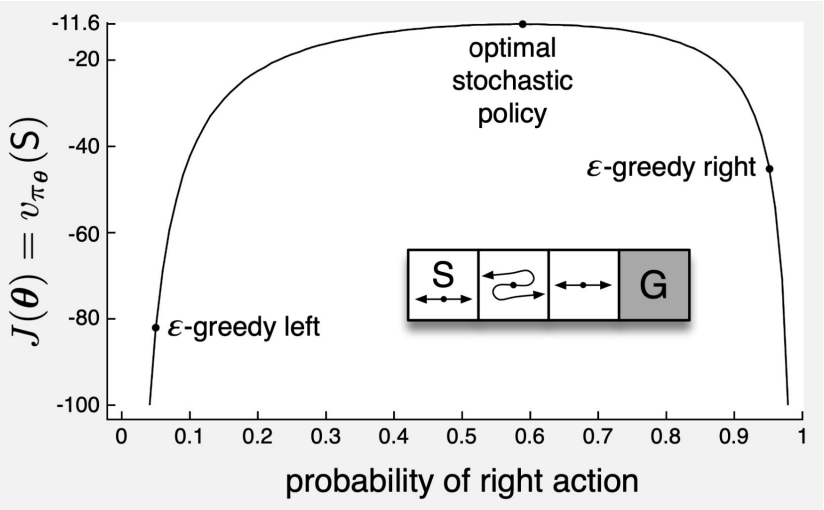
$$\mathbf{x}(s, \text{right}) = [1, 0]^\top$$
$$\mathbf{x}(s, \text{left}) = [0, 1]^\top$$

- Milestone question: What is a good (optimal) policy?

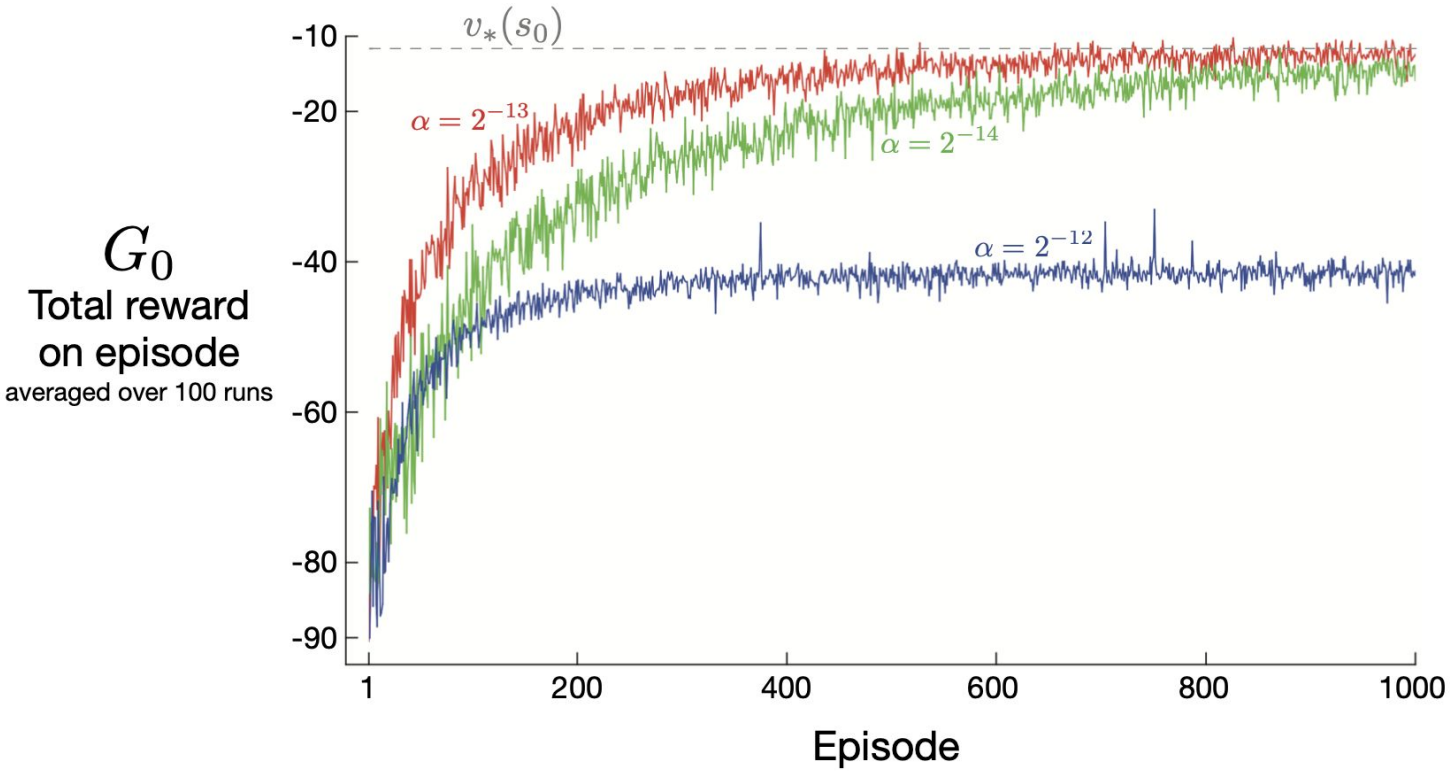
Example: Short corridor with switched actions

- With a policy gradient method, we should be able to find the stochastic policy: P(RIGHT) = 0.59
- An action-value method with "epsilon-greedy action selection is forced to choose between just two policies
  - choosing right with high probability 1 - eps/2 or
  - choosing left with the same high probability
- The plot shows the values based on eps = 0.1





REINFORCE on the Short Corridor



Notes on REINFORCE

- By construction, the expected update over an episode is in the same direction as the performance gradient
- This assures
  - an improvement in expected performance for sufficiently small step size (alpha)
  - convergence to a local optimum under standard stochastic approximation conditions for decreasing step size
- However, as REINFORCE may be of high variance (due to the MC concept) and thus produce slow learning
- We will see an improvement in the next slideset (#08)

Coding Exercise